



Are Requirements Alive and Kicking?

Jane Cleland-Huang

ABOUT 10 YEARS ago, I attended a large meeting for requirements engineers and business analysts in Europe. At that time, one of the organizers asked how many of us thought agile methods would significantly impact the way mainstream companies developed software and managed requirements. None of us raised our hands.

In hindsight, of course this seems particularly shortsighted. In our defense, most of the people in that room had invested a significant amount of time developing techniques for improving the requirements process. We knew how important it was to identify relevant stakeholders, proactively elicit requirements, analyze them carefully,

time, I coauthored a book with Mark Denne, then from Sun Microsystems, called *Software by Numbers: Low Risk, High Return Development* (Prentice Hall, 2003), in which we laid out a highly incremental, ROI-driven approach to prioritizing and sequencing features. What I didn't then realize is that we had serendipitously discovered an idea that fit naturally into the agile development process. Our approach still assumed that we would identify stakeholders and work with them to discover their needs, but instead of laying out a long-term development plan, it used a simple financial analysis to figure out which features (architectural or functional) to build first. Furthermore, the

to discuss the challenges that impact requirements processes today and the context in which most of us deal with requirements. Agility is certainly one of the major influences.

Changing Times

As an academic, I've had the opportunity to engage in industrial projects of many sizes. Some of these projects are very linear in nature and basically follow the typical waterfall model, whereas others are agile and fully embrace concepts such as informal user stories, short delivery cycles, and changing requirements. From my perspective, both kinds of projects have had their own challenges, but in the end, both kinds of projects have been successful.

The bottom line is that the IT environment we work in today is simply different from the environment 10 years ago, and for this column to continue to address pertinent topics, it's essential that we understand the critical forces that drive requirements today.

For example, we can no longer assume that projects will be preceded by stringent, up-front processes in which we carefully elicit all the requirements and then design a complete solution before jumping into the actual development process. There are many assumptions that we might have made a decade ago that simply aren't true today.

We can no longer assume that it's possible to bring all stakeholders

I came to realize how requirements and agility could live side by side.

identify trade-offs, emerge and negotiate conflicts, and specify clearly and unambiguously what the system needed to do and how it needed to be done. At that time, the agile movement seemed to be telling us to throw these practices away, threatening the whole concept of requirements as we knew it.

On the other hand, around the same

approach embraced change by allowing the upcoming set of features to be determined as the project progressed. Consequently, I came to realize how requirements and agility could live side by side.

So, why am I focusing on agility in a column on requirements? For my debut installment of this column, I want

together in the same room at the same time for face-to-face meetings. In many current projects, stakeholders are distributed around the globe, and sometimes the only way to reach them all is by leveraging social networking and collaboration tools. This introduces numerous challenges. How can we make our tools more effective so that meetings made up of people from different continents produce meaningful discussions that bring forth real issues and result in a deep understanding of stakeholder needs?

We're gaining a far deeper understanding of the synergies between requirements and architectural design. In fact, in many cases, new requirements are introduced to an existing system, and previous architectural, platform,

and hardware decisions create constraints on the viability of new feature requests. Although it's true that almost anything is possible, it's also true that some requirements are far easier to deliver than others and that existing solutions can facilitate or hinder future change. So, how does this more incremental life cycle affect the way we elicit and manage requirements?

Software development environments (not just the stakeholders) are becoming increasingly global in nature. When the outsourcing trend began a decade or so ago, we had fairly clear hand-off points in the life cycle. That line is far fuzzier now. Outsourcing companies are now often responsible for not just coding and testing but also the requirements elicitation process. We must con-

sider how this affects project success and how we can ensure that project goals and requirements are fully explored and understood when the people eliciting the requirements aren't even on the same continent as the primary stakeholders.

New Demands

There's also a move to push up the abstraction level of the software development effort by specifying software requirements as models and then automatically generating code from those models. In the world of model-driven development (MDD), how do we ensure that requirements are specified correctly in models and verify that the generated code actually satisfies stakeholders concerns?



So you are already a
Certified Professional for
Requirements Engineering at
Foundation Level, then you have the
solid basis you need for growth.

Go for the CPRE Advanced Levels

- **Requirements Elicitation and Consolidation**
Get the right skills to obtain all the facts from stakeholders and to separate the wheat from the chaff!
- **Requirements Modeling**
Acquire the best modeling techniques and information will never ever slip through your fingers!

www.ireb.org *The home of Requirements Engineering*

IEEE SOFTWARE CALL FOR PAPERS

Green Software

Submission deadline: 25 June 2013 • Publication: Jan./Feb. 2014

Information technologies (IT) requiring vast amount of energy and other resources are used in almost every field and process. Green IT is the study and practice of using computing resources efficiently to reduce negative impacts on the environment. Green IT is applicable to various high-tech domains, such as datacenters, mobile computing, and embedded systems. Recently, global carbon dioxide emissions reached 9.1 billion tons, the highest level in human history—49 percent higher than in 1990 (the Kyoto reference year). At least 2 percent of global carbon dioxide emissions can be attributed to IT systems, and further increases are expected, with new IT systems being deployed daily. Therefore, reducing the energy consumption and related carbon dioxide emission of IT systems is a crucial requirement. Reducing energy consumption also leads to reduced maintenance expenses and costs of ownership, giving manufacturers a competitive advantage.

Questions?

For more information about the focus, contact the guest editors:

- Ayse Basar Bener, Ryerson University: ayse.bener@ryerson.ca
- Maurizio Morisio, Politecnico di Torino: maurizio.morisio@polito.it
- Andriy Miranskyy, IBM Toronto Software Lab: andriy@ca.ibm.com

Full call for paper: www.computer.org/software/cfp1

Full author guidelines: www.computer.org/software/author.htm

Submission details: software@computer.org

Submit an article: <https://mc.manuscriptcentral.com/sw-cs>

Yet another challenge comes from the increasing desire for systems, especially Web services, to adapt at runtime to changes in their environments. One of my graduate students recently told me that his company's attempt to achieve adaptation failed primarily because it didn't know how to clearly specify requirements for adaptive systems, and therefore never fully understood the system's goals. As industry increasingly moves toward building adaptive systems, we must find better ways to specify adaptation goals.

And what about safety-critical software development? In critical systems, it's particularly important to perform a rigorous risk analysis, identify hazards and faults, and then specify mitigating requirements, which are carefully tracked throughout the system's development. How should the requirements process be conducted so that the end result is a carefully constructed assurance or safety case that demonstrates a system is safe for use? What

requirements techniques are effective in these kinds of environments?

Although the IT environment has changed over the years, the importance of eliciting and understanding requirements is timeless. The end result may be represented in different ways—as traditional “shall” statements, use cases, sketches, user stories, acceptance tests, formal logic, goal models, or state charts. However, as long as we continue to build software systems and care whether those systems meet the needs of our customers and other stakeholders, then we must continue to emphasize the importance of understanding, analyzing, and specifying requirements.

In taking over this column from Neil, I realize that I have some metaphorically large shoes to fill. Neil brought us eight years of wisdom, debate, and sometimes hilarity in the form of Colin Codephirst. I'm not even going to try to fill those shoes. Instead,

I hope we can go on a new journey that winds its way through new challenges that emerge along the way. I hope to discuss current issues related to requirements and to take on some of the most controversial issues head on.

We must continue our quest to learn better ways to work with stakeholders to discover requirements, embrace change, deliver safe systems, engage distributed stakeholders, and support innovation and change. Although the challenges are endless, the benefits are immense. Requirements are very much alive and kicking!

If you have ideas for requirements-related topics that you would like to see discussed, please email me at jhuang@cs.depaul.edu. 

JANE CLELAND-HUANG is an associate professor at DePaul University. Contact her at jhuang@cs.depaul.edu.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

IEEE SOFTWARE CALL FOR PAPERS

Special Issue on Next Generation Mobile Computing

Submission deadline: 30 June 2013 • Publication: Mar./Apr. 2014

Ubiquitous, pervasive mobile computing is all around us. We use mobile computing not only when we interact with our smartphones to connect with friends and family across states and countries, but also when we use ticketing systems on a bus or train to work or home, purchase food from a mobile vendor at a park, watch videos and listen to music on our phones and portable music playing devices. In other words, mobile computing is not only the interaction of smart phones with each other. Any computation system that is expected to move and interact with end users or other computational systems despite potential changes in network connectivity—including loss of connectivity or changes in type of connectivity or access point—participates in mobile computing infrastructure, and the number of such systems is expected to grow significantly each year over the coming decades.

Questions?

For more information about the focus, contact the guest editors:

- James Edmondson, Carnegie Mellon Software Engineering Institute: jredmondson@sei.cmu.edu
- William Anderson, Carnegie Mellon Software Engineering Institute
- Joe Loyall, BBN
- Jeff Gray, University of Alabama
- Jules White, Virginia Tech
- Klaus Schmid, University of Hildesheim

Full call for paper: www.computer.org/software/cfp2

Full author guidelines: www.computer.org/software/author.htm

Submission details: software@computer.org

Submit an article: <https://mc.manuscriptcentral.com/sw-cs>